AI FOR CODERS AND SOFTWARE ENGINEERS · DAY 2

Generative AI in Code

From a prompt in Copilot to your own AI applications in Python

OpenAI + system prompt | Gradio app | RAG with LlamaIndex | SQL data exploration

Instructor: Mohammad Arshad Ahmed

WHERE WE ARE

The four-day journey

Day 1

Copilot and prompts

Prompt engineering, data understanding, A-001 investigation. No code.

Day 2 · today

GenAI in code

OpenAI from Python, system prompts, Gradio, RAG with LlamaIndex, SQL.

Day 3

Agentic AI

An agent that uses tools: your RAG, your SQL, a decision tree.

Day 4

Capstone

Team project, governance, demos.

Today's prompt becomes tomorrow's system message. Today's RAG and SQL become tomorrow's agent tools.

TODAY

Three labs, one story

Lab 1 — 45 min

OpenAI + system prompt + Gradio

Call a reasoning model, add a system prompt, set reasoning effort, build a chat app.

`Day2_01_OpenAI_System_Prompt_and_Gradio.ipynb`

Lab 2 — 60 min

RAG with LlamaIndex

Upload 3 to 4 documents, chunk, embed, search top-k = 3, answer with sources.

`Day2_02_RAG_with_LlamaIndex.ipynb`

Lab 3 — 60 min

SQL EDA and dashboard

Upload the database, explore with SQL, build a one-page dashboard.

`Day2_03_SQL_EDA_and_Dashboard.ipynb`

All in Google Colab. You upload the files from ENEC2026_Day2_Upload_Pack.zip. No GitHub, no installs.

BEFORE WE START

Five-minute setup

1 Open Colab

colab.research.google.com, signed in with a Google account.

2 Add your key to Secrets

Key icon on the left > Add new secret > Name: OPENAI_API_KEY > Notebook access ON.

3 Unzip the Day 2 pack

rag_documents/ (4 files for Lab 2) and database/ENEC2026_Asset360.db (Lab 3).

4 Open the notebook

File > Upload notebook, then choose the Lab 1 notebook.

5 Run top to bottom

Shift + Enter on each cell. Errors: read the last line first.

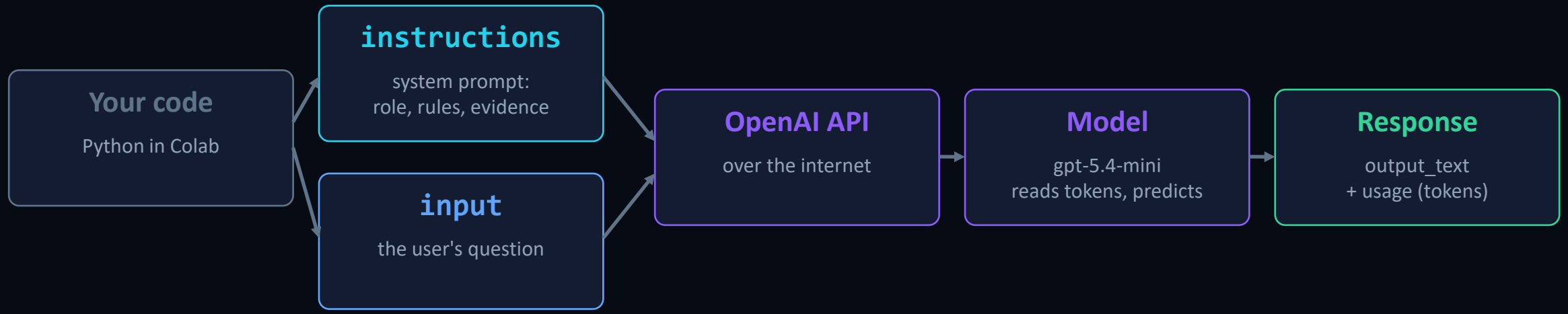
PART 1

Talking to a model in code

Tokens, system prompts, reasoning effort and a Gradio app

HOW IT WORKS

One request, in code




```
response = client.responses.create(
    model="gpt-5.4-mini",
    instructions=SYSTEM_PROMPT,     # the system prompt
    input="What is the health score of A-001?",
    reasoning={"effort": "low"},
)
print(response.output_text)
```

How words become tokens

"Pump A-001 vibration is rising"



␣ = a space. The space is part of the token. Example IDs from the GPT-4 tokenizer; the GPT-5 tokenizer in your notebook gives different numbers.

5 words → 8 tokens

The model reads, and you pay for, tokens, not words.

≈ 4 characters per token

100 tokens ≈ 75 English words. A page ≈ 500 to 700 tokens.

Rare words split more

"unbelievably" → un | belie | vably. Codes like A001-PROC-INS-001 cost many tokens.

Why tokens matter

Context window

400,000 tokens for gpt-5.4-mini (prompt + answer together). About 300,000 words.

Cost

You pay per token: input + output. Reasoning tokens are hidden but billed as output.

Speed

More tokens in and out means slower answers. Short, focused prompts are faster.

4 documents ≈ 3,200 tokens

Paste everything into every prompt? Fine for 4 documents. Not for 4,000. RAG sends only the 3 most relevant chunks: cheaper, faster, and the model is not lost in a haystack.

SYSTEM PROMPT

The system prompt is the model's job description

`System prompt → instructions=`

ROLE You are a reliability engineering assistant at a power plant.

RULES Use ONLY the evidence. If it is not there, say "Not in the data". Never approve shutdowns.

FORMAT One-line answer, then up to three bullet points.

EVIDENCE asset_id: A-001 | health_score: 68.05 | risk_level: MEDIUM | recommended_action: Increase monitoring

`User prompt → input=`

"What is the health score of A-001 and what should we do next?"

Stays vs changes

The system prompt stays the same for every question. The input changes each time.

Day 1 Copilot prompt = Day 2 system prompt. Same skill, now in code.

Same model, same question, three results

1 · No evidence

Q: "What is the health score of A-001?"

Guesses, or says it cannot know. It has never seen our plant.

2 · System prompt + evidence

Same question, with the record in the system prompt

"Health score 68.05, risk MEDIUM. Recommended action: increase monitoring."

3 · Question outside the data

Q: "Who made the pump and what does a new one cost?"

"Not in the data." No invented manufacturer, no invented price.

Ground it with evidence, and give it permission to say "I don't know".

Reasoning effort: how hard the model thinks



faster, cheaper

deeper, slower, more tokens

low

Quick facts, summaries, rewriting, answering from documents (RAG).

medium

Normal analysis: compare options, explain a trend.

high

Multi-step problems, planning, tricky reasoning.

Reasoning tokens are hidden but billed as output.
Reasoning models ignore "temperature". Effort is the knob.

```
client.responses.create(
    model="gpt-5.4-mini",
    input=question,
    reasoning={"effort": "low"},
)
```

GRADIO

From a Python function to a web app

1

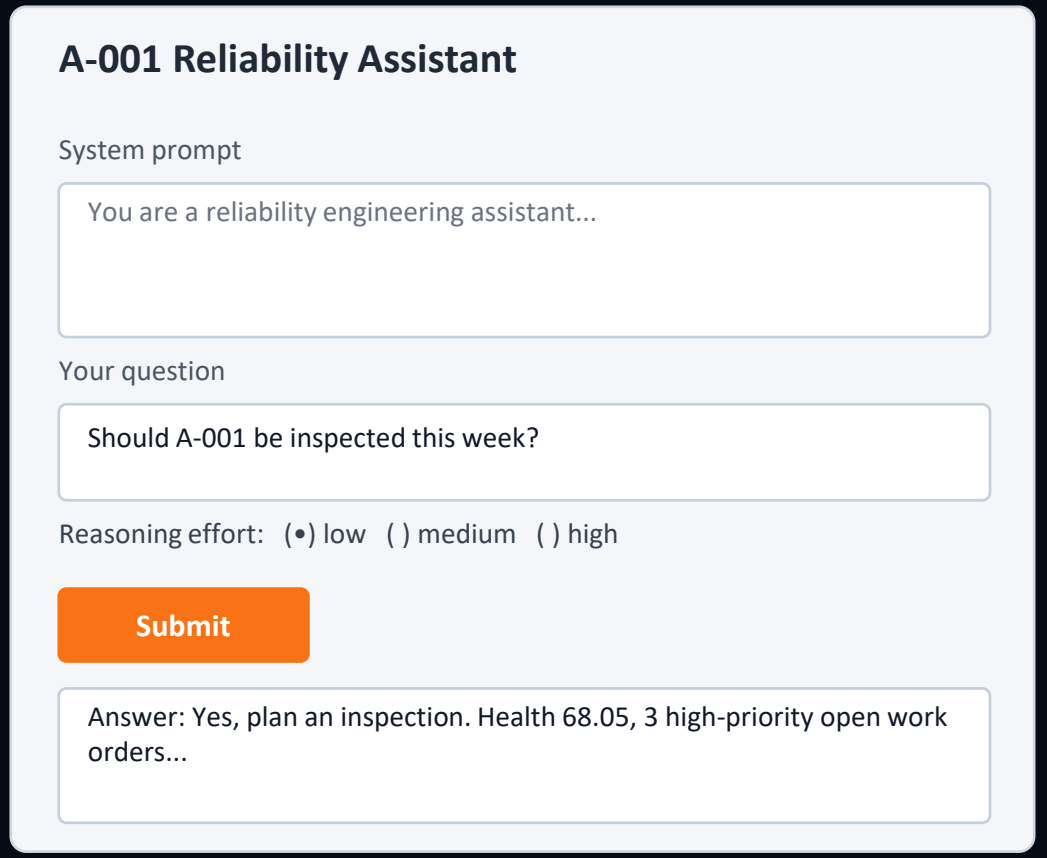
```
def chat_app(system_prompt, question, effort):
    return ask(question, system_prompt, effort)
```

2

```
demo = gr.Interface(
    fn=chat_app,
    inputs=[gr.Textbox(label="System prompt"),
            gr.Textbox(label="Your question"),
            gr.Radio(["low", "medium", "high"])],
    outputs=gr.Markdown(label="Answer"),
)
```

3

```
demo.launch()   # appears under the cell in Colab
```



Change the system prompt in the app, ask the same question, and watch the answer change.

LAB 1

OpenAI, system prompt and Gradio · 45 minutes

1 **Steps 1 to 4**

Key in Secrets, install, connect, first question.

2 **Step 5 · Tokens**

Tokenize your own sentence. Count words vs tokens.

3 **Steps 6 to 7 · System prompt**

Ask without evidence, then with the system prompt. Compare.

4 **Step 8 · Reasoning effort**

Same question at low, medium, high. Time and reasoning tokens.

5 **Steps 9 to 10 · Gradio**

One function, one gr.Interface, demo.launch().

6 **Exercises**

Remove the safety rule. Change the format. Write a system prompt for your own job.

PART 2

RAG: chat with your documents

Chunking, embeddings, vector stores, semantic search and top-k with LlamaIndex

The model does not know your documents

The problem

Trained on public data up to a cut-off date. Never saw your procedures, reports or work orders. Ask about them and it guesses.

Option 1 · Paste everything

Every document in every prompt.
Expensive, slow, and the model loses the needle in the haystack. Does not scale past a few files.

Option 2 · RAG

Retrieve only the 3 most relevant pieces, then generate the answer from them, with sources a person can check.

RAG = Retrieval-Augmented Generation = first RETRIEVE the right pieces, then GENERATE the answer.

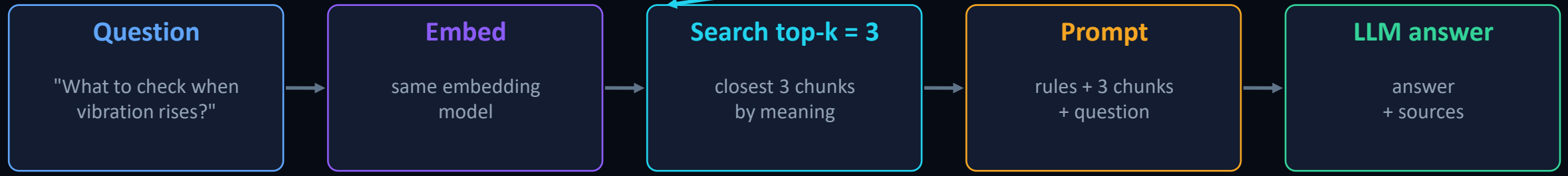
Two phases: build the index once, then ask many questions

PHASE 1 · INDEX (once)



search the vector store

PHASE 2 · QUERY (every question)



The model only ever sees the 3 retrieved chunks, never the whole library.

Cut documents into pieces the search can find

One document

Scope and purpose

Safety precautions

Vibration limits

Inspection steps

Records and sign-off



- chunk 1 · Scope and purpose
- chunk 2 · Safety precautions
- chunk 3 · Vibration limits
- chunk 4 · Inspection steps
- chunk 5 · Records and sign-off

Our 4 documents (≈ 3,200 tokens), overlap 50

`chunk_size = 200` precise, may lose context	**29 chunks**
`chunk_size = 400` our setting today	**11 chunks**
`chunk_size = 800` more context, vaguer search	**7 chunks**

Too small: pieces lose meaning. Too big: vague search and wasted tokens. Start at 400, then test.

Overlap keeps ideas that cross a border

Without overlap

...pump A-001 vibration rose above 4.5 mm/s

on 18 August, and anomaly minutes began...

Chunk 1 and chunk 2 each hold half a fact. The search can miss it.

With overlap (50 tokens)

chunk 1: ...pump A-001 **vibration rose above 4.5 mm/s**

50 / 400 ≈ 12%

Typical: 10 to 20% of the chunk size.
LlamaIndex default: 1,024 / 200.

chunk 2: **vibration rose above 4.5 mm/s** on 18 August, and anomaly minutes began...

amber = the overlap, repeated in both chunks. Chunk 2 now holds the complete fact.

Turning meaning into numbers

"The pump vibration is rising"

↓ text-embedding-3-small

```
[ 0.0123, -0.0441, 0.0870, 0.0036,
  -0.0219, 0.0512, ... ]

1,536 numbers = 1 vector
```

Similar meaning → vectors close together

No word "vibration", but it lands next to the vibration sentences.



Where the chunks and their vectors live

id	chunk text	vector (1,536 numbers)	metadata
1	"V2 procedure: if vibration exceeds the alert level, increase monitoring..."	`[0.012, -0.044, 0.087, ...]`	file: 04_..._V2_Approved
2	"Rising vibration: check alignment, bearing condition, cavitation..."	`[-0.031, 0.020, 0.064, ...]`	file: 06_..._Troubleshooting
3	"Daily average vibration rose from about 1.4 mm/s in late July..."	`[0.008, -0.052, 0.071, ...]`	file: 07_..._Condition_Report
...	11 chunks in total for our 4 documents	...	...

Metadata = sources

The file name travels with every chunk, so every answer can cite its document.

Today: in memory

VectorStoreIndex keeps it in memory. Production: a vector database (Pinecone, Chroma, pgvector).

SEMANTIC SEARCH

How top_k = 3 works

"What should be checked when vibration on A-001 is rising?"

```
question vector
[0.021, -0.038, 0.079, ...]
```

cosine similarity:
1.0 = same meaning, 0 = unrelated

top_k = 3

0.71	06 Troubleshooting guide · rising vibration checks
0.66	07 Condition report · vibration trend July to August
0.58	04 V2 procedure · alert and alarm levels
0.41	09 Field report · wet area near pump
0.33	04 V2 procedure · records and sign-off
0.27	07 Condition report · scope

Illustrative scores. Lab 2, step 9 prints the real ones.

Only the top 3 chunks go to the LLM. k too small: missed evidence. k too big: noise and cost.

Search by meaning, not by matching words

Q: "Why is the pump shaking more than usual?"

Keyword search (Ctrl+F)

Looks for the word "shaking".
The documents say "vibration".

0 results.

Semantic search (embeddings)

Compares meaning.
"shaking" ≈ "vibration".

Finds: "Vibration on the pump has increased over the last four weeks."

Lab 2, step 7: one question, three sentences, compare the similarity scores yourself.

The whole RAG pipeline in a few lines

```
Settings.llm = OpenAI(model="gpt-5.4-mini", reasoning_effort="low")
Settings.embed_model = OpenAIEmbedding(model="text-embedding-3-small")
```

1 **Settings: model, embeddings, chunking**

```
documents = SimpleDirectoryReader("/content/my_documents").load_data()
```

2 **Load the files you uploaded**

```
nodes = SentenceSplitter(chunk_size=400,
chunk_overlap=50).get_nodes_from_documents(documents)
```

3 **Chunk**

```
index = VectorStoreIndex(nodes)
```

4 **Embed + vector store**

```
retriever = index.as_retriever(similarity_top_k=3)
```

5 **Semantic search, top 3**

```
query_engine = index.as_query_engine(similarity_top_k=3)
response = query_engine.query("What should be checked...?")
```

6 **Retrieve + generate + sources**

LLAMAINDEX SETTINGS

Six settings that shape your RAG system

`Settings.llm`

gpt-5.4-mini · reasoning_effort="low"

Writes the answer from the chunks. Higher effort = slower, rarely needed for RAG.

`Settings.embed_model`

text-embedding-3-small · 1,536 dimensions

Question and chunks must use the SAME embedding model.

`Settings.chunk_size`

400 tokens

Smaller = more, precise chunks. Bigger = fewer, broader chunks.

`Settings.chunk_overlap`

50 tokens (≈ 12%)

Keeps sentences that cross a chunk border.

`similarity_top_k`

3

How many chunks reach the LLM. Try 1 and 5 and compare.

`system_prompt`

Answer only from the documents

Say "The documents do not say" when the answer is missing. Name the source.

LAB 2

RAG with LlamaIndex · 60 minutes

1 Upload 4 files

04 V2 Approved Procedure · 06 Troubleshooting Guide · 07 Condition Monitoring Report · 09 Field Condition Report

2 Steps 4 to 5 · Settings and load

Print every setting. Load the folder.

3 Step 6 · Chunks

Read 2 chunks. Compare 200 / 400 / 800.

4 Step 7 · Embeddings

1,536 numbers. "shaking" vs "vibration" test.

5 Steps 8 to 9 · Index and search

VectorStoreIndex, retriever top_k = 3: scores and files.

6 Steps 10 to 13 · Ask and app

Answers with sources, with vs without RAG, Gradio app.

Exercises: chunk_size 200 · top_k 1 and 5 · upload the security documents and build a new assistant.

When RAG goes wrong, and what to check

Wrong chunk retrieved

Print the top-k chunks first. Then tune chunk_size or top_k.

Answer not in the documents

Allow "The documents do not say". Test with a question you know is missing.

Old and draft versions

Search cannot tell approved from draft. Index only approved documents, or add status metadata and rules.

No sources, no trust

Always show the files behind the answer. A person checks and decides.

PART 3

Know your data with SQL

Exploratory data analysis on the plant database, then a dashboard

Five SQL ideas cover most questions

SELECT … FROM	`SELECT asset_id, health_score` `FROM asset_360`	choose columns and table
WHERE	`SELECT * FROM work_orders` `WHERE status IN ('OPEN','IN_PROGRESS')`	keep matching rows
GROUP BY	`SELECT criticality, COUNT(*)` `FROM asset_360 GROUP BY criticality`	one row per group
ORDER BY · LIMIT	`SELECT asset_id, health_score FROM asset_360` `ORDER BY health_score LIMIT 10`	sort, keep the top
JOIN	`FROM work_orders w JOIN asset_360 a` `  ON a.asset_id = w.asset_id`	combine tables by asset_id

LAB 3

SQL EDA and a dashboard · 60 minutes

1 Upload ENEC2026_Asset360.db

into /content/database

2 Connect and list 24 tables

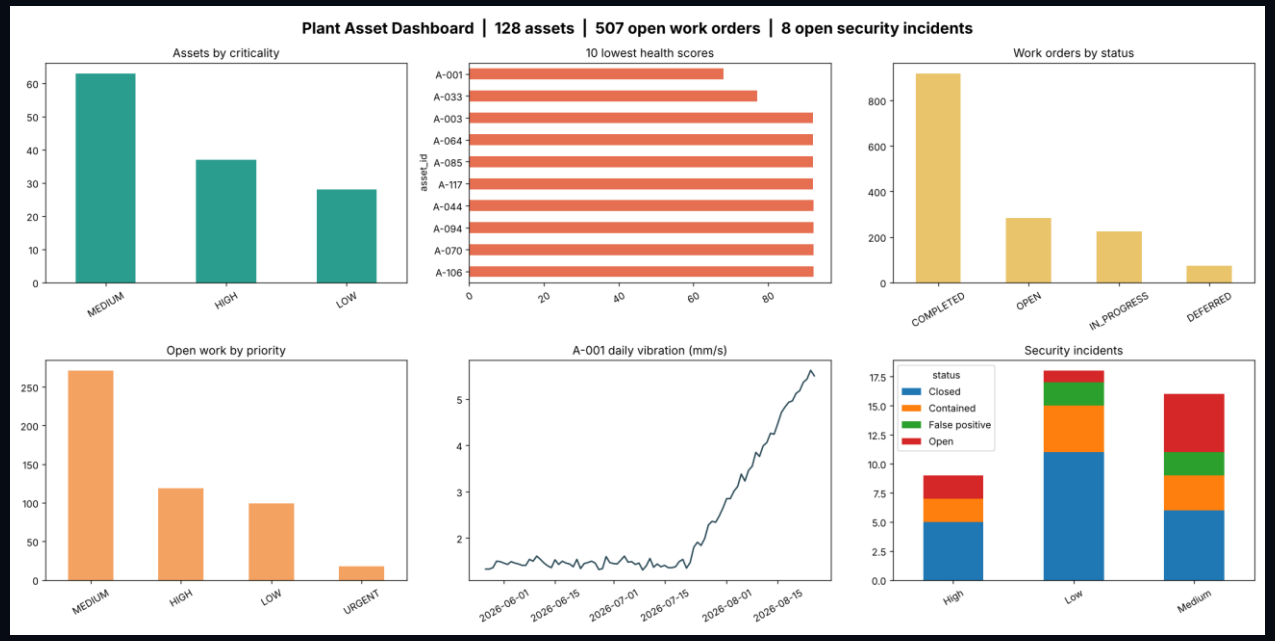
all linked by asset_id (A-001 to A-128)

3 Explore with SQL

columns · missing values · criticality · lowest health · open work · JOIN · A-001 vibration · incidents

4 Build the dashboard

6 charts + 3 headline numbers on one page



A-001: health 68.05, far below every other asset. Vibration flat at ~1.4 mm/s, rising from late July to 5.5 mm/s.

WRAP-UP

What you built today, and where it goes tomorrow

System prompt + Gradio

Your structured prompt in code, in a web app.

RAG with LlamaIndex

Chunk, embed, store, search top-k = 3, answer with sources.

SQL EDA + dashboard

The plant data, explored and on one page.

Day 3 · Agentic AI

Agent = LLM that chooses tools

RAG tool

documents

SQL tool

data

Decision tree

prediction

Tonight: keep your key in Colab Secrets and keep the Day 2 pack. We reuse both tomorrow.